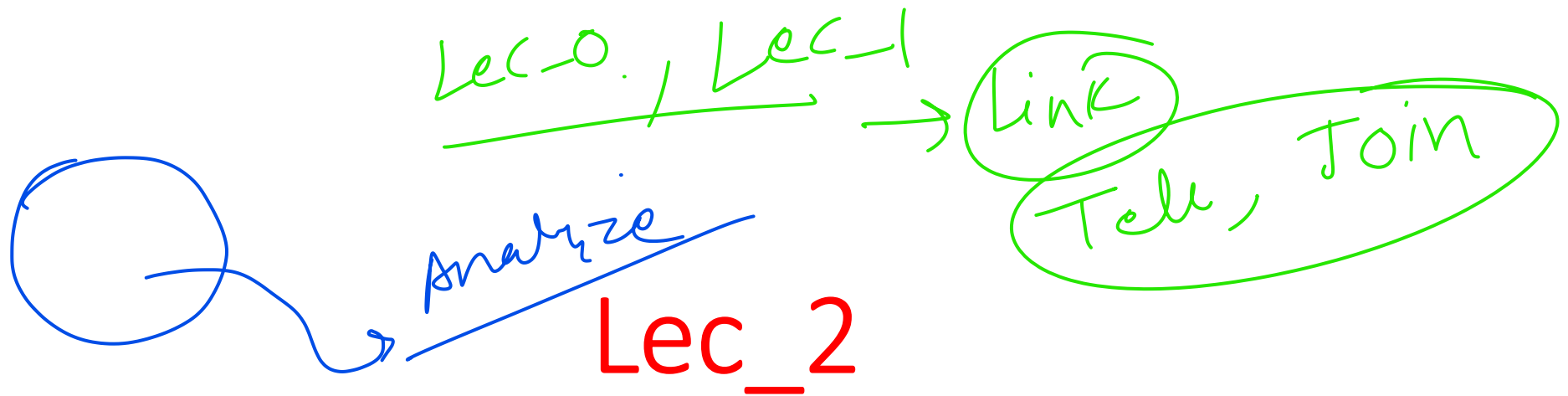




Analysis of Algorithms

A problem can have more than one algorithm, so best algo is decided by 2 factors



pseudo code algorithm rule $\Rightarrow$ program

2.5ms

hello

C++, C, Java, Python

```
#include < >
main()
{
    cout << "hello" << endl;
}
```

Compile + Run
1 2

Time complexity

- The time complexity of an algorithm is the amount of time it takes for each statement to complete

$$T(P) = C(P) + R(P)$$

- $C(P)$ = Compile time / depends on language of the compiler
- $R(P)$ = Processor time (Hardware), depends on type of processor

$$T(P) = C(P) + R(P) \\ = \text{Compile} + \text{Run}$$

processor $\Rightarrow x \rightarrow$
" $\Rightarrow 2x$

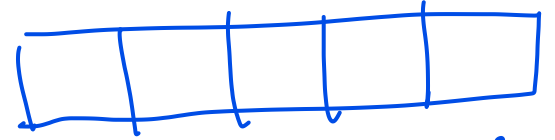
main
{
-
-
-
-
-
}
3

Time C
compile
 $\downarrow$
compiler (software)
 $\downarrow$
prog. l.

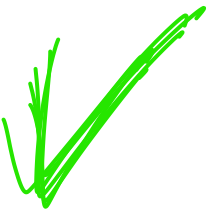
Space Complexity :-

Amount of memory required to run a program is called space complexity.

=> Array



=> To create an array of the size n ~~in~~ dots $\rightarrow$ Byte
 $2n$ Byte



program $\downarrow$ complete

Analyze

Plan
Theoretical approach

Compile + Run $\Rightarrow$ than analyze

Plan 2
(0.4 ms)

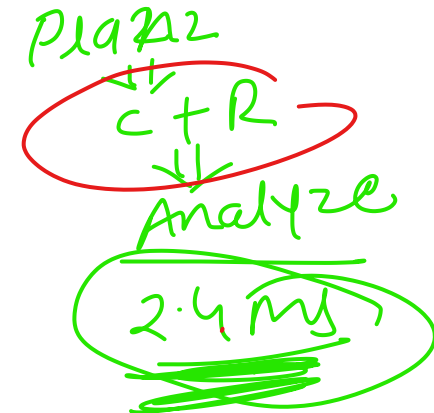
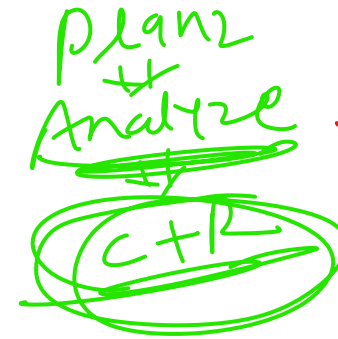
Analysis of an Algorithm

• Posteriori

plan2

• Priori

plan1



1.

It is dependent on language of compiler and type of hardware.

It is independent of language of compiler and types of hardware.

2.

It will give exact answer.

It will give approximate answer.

3.

It doesn't use asymptotic notations to represent the time complexity of an algorithm.

It uses the asymptotic notations to represent how much time the algorithm will take in order to complete its execution.

$\{O, \Omega, \Theta\}$
 $\{n, \log n, \dots\}$

4.

The time complexity of an algorithm using a posteriori analysis differ from system to system.

The time complexity of an algorithm using a priori analysis is same for every system.

Analyze

Q.

It is dependent on language of
compiler and type of hardware.

It will give exact answer.

It is independent of language of
compiler and types of hardware.

It will give approximate answer.

Dhyan do ★
important
gave questions

Plan1

Priori Analysis:

$\Rightarrow$

Rules

$\Rightarrow$ theoretical method

$\rightarrow$ no exact Answer

Post-run Plan2 {
2.6 ms

✓ Notations with Examples

1. Big-O Notation (O-notation)

2. Omega Notation (Ω -notation)

3. Theta Notation (Θ -notation)

Big O $\Rightarrow$ max^m time = $O(n)$
min
avg $\Omega(n)$
 $\Theta(n)$

$(n-2) \times$

max^m = n times

Q.7/ Write a program to print
1 to n numbers?

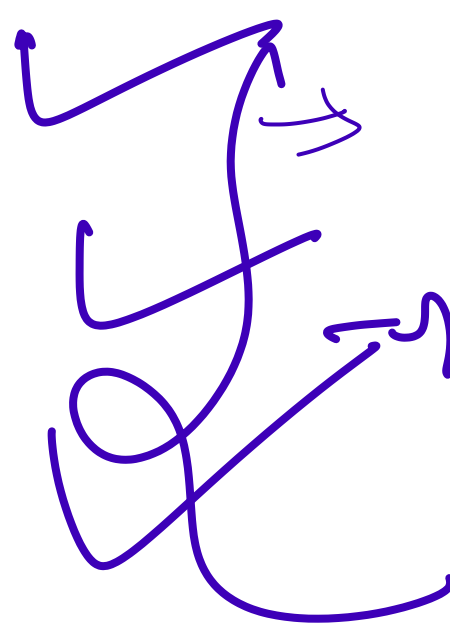
```
#include <stdio.h>
main()
```

```
{
    int n;
    scanf("%d", &n);
    for (int i = 1; i <= n; i++)
    {
        printf("%d", i);
    }
}
```

loop

1, 2, 3, 4, 5, n

⇒ n times

 Big O $\Rightarrow$ $\underline{\underline{O, \omega}}$ Maximum, case
Big Ω $\Rightarrow$ Minimum case
Big Θ $\Rightarrow$ Average case

' => Loop, Array & LinkedList

"Thanks for watching"

